Ironclad Mobile Security SDK: RASP & Defense-in-Depth

Master Technical Handbook: Beginner to Extreme Senior Architecture

Project: Ironclad Flutter Security SDK
Engineer: Akshay Chandran P
Tech Stack: Flutter, Dart, Kotlin, Swift, Platform Channels, RASP, Hardware Keystore, Secure Enclave, SPKI Cert Pinning

---

Section 1: End-to-End Architecture & Working Explanation

1.1 Runtime Application Self-Protection (RASP)

Mobile apps operate in hostile zero-trust environments where attackers can run debuggers, inspect memory, hook functions via **Frida**, or repackage APKs.
Ironclad provides enterprise-grade defense-in-depth: 1. **Root / Jailbreak Detection:** Checks for binary paths (`/system/xbin/su`, `/Applications/Cydia.app`), Magisk mount points, test-keys builds, and attempts to execute root shell commands. 2. **Anti-Hooking & Frida Detection:** Inspects `/proc/self/maps` for Frida libraries, scans open TCP ports (`27042`), and verifies symbol table integrity. 3. **Anti-Debugging:** Invokes `ptrace(PTRACE_TRACEME)` on Linux/Android and `sysctl` on iOS to prevent debugger attachment. 4. **Certificate Pinning:** Pins the Subject Public Key Info (SPKI) SHA-256 fingerprint, defeating proxy interception tools (Charles, Burp Suite).

1.2 Hardware Keystore AES-256 Envelope Encryption

To protect sensitive tokens and database keys: 1. App requests a Master Key from Android Keystore / iOS Secure Enclave. 2. Master Key is backed by dedicated tamper-resistant hardware chips (Titan M, Apple Secure Enclave). 3. Data is encrypted using AES-256-GCM with a unique 96-bit initialization vector (IV) per record. 4. The Master Key never leaves the physical hardware enclave.

(Handbook continues with pub.dev package structuring, platform channel C++ implementations, and security compliance)